TD Chapitre 3 : Structures et Unions

Programmation C Avancée - L3

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

Contexte

Architecture x86-64, little-endian. Alignement: `char`=1, `short`=2, `int`=4, `long`/pointeur=8, `double`=8. Tous les dumps hexadécimaux utilisent 2 caractères par octet.

1 Exercice 1 : Calcul de Padding

Pour chaque structure, calculer: (a) l'offset de chaque membre, (b) le padding inséré, (c) `sizeof` total.

1.1

```c
struct A {
    char a;
    int b;
    char c;
};
```

1.2

```c
struct B {
    char a;
    char b;
    int c;
    short d;
};
```

1.3

```c
struct C {
    double x;
    char tag;
    int count;
    char flag;
};
```

1.4

```c
struct D {
    char a;
    short b;
    char c;
    long d;
    char e;
};
```

2 Exercice 2 : Analyse de Dumps Hexadécimaux

2.1 - Reconstruction de structure

Le dump suivant correspond à une variable de type `struct Data` à l'adresse `0x7FFE1000`:

```
0x7FFE1000: 41 00 00 00 78 56 34 12 42 43 00 00
```

```c
struct Data {
    char tag;
    int value;
    short code;
};
```

Questions:

1. Identifier les octets de données et de padding
2. Reconstruire les valeurs: `tag`, `value`, `code`
3. Quel est le `sizeof(struct Data)` ?

2.2 - Structure imbriquée

```
0x7FFE2000: 0A 00 00 00 14 00 00 00 1E 00 00 00 28 00 00 00
            52 00 00 00 00 00 00 00
```

```c
struct Point { int x, y; };
struct Rectangle {
    struct Point top_left;
    struct Point bottom_right;
    char color;
};
```

Questions:

1. Identifier chaque champ dans le dump
2. Extraire les coordonnées des deux points
3. Quelle est la valeur de `color` (code ASCII) ?
4. Combien d'octets de padding final ?

2.3 - Tableau de structures

```
0x7FFE3000: 41 00 64 00 42 00 C8 00 43 00 2C 01
```

```c
struct Entry {
    char id;
    short value;
};
struct Entry entries[3];
```

Questions:

1. Quel est le `sizeof(struct Entry)` ?
2. Reconstruire le tableau `entries`
3. Quel est l'offset de `entries[2].value` ?

3 Exercice 3 : Optimisation du Padding

3.1 - Réorganisation

```c
struct Original {
    char a;
    double b;
    char c;
    int d;
    char e;
};
```

Questions:

1. Calculer `sizeof(struct Original)`
2. Proposer une réorganisation minimisant la taille
3. Calculer la nouvelle taille
4. Économie réalisée en octets et en pourcentage

3.2 - Structure avec pointeurs

```c
struct Node {
    char type;
    struct Node *next;
    int data;
    char flags;
};
```

Questions:

1. Calculer `sizeof(struct Node)` (x86-64)
2. Proposer une version optimisée
3. Quelle taille pour la version optimisée ?

4 Exercice 4 : Unions

4.1 - Interprétation mémoire

```c
union U {
    int i;
    float f;
    char c[4];
};

union U u;
u.i = 0x40490FDB;
```

Questions:

1. Quel est le dump hexadécimal de `u` (little-endian) ?
2. Quelle est la valeur de `u.c[0]`, `u.c[1]`, `u.c[2]`, `u.c[3]` ?
3. `0x40490FDB` est la représentation IEEE 754 de π. Quelle sera la valeur (approximative) de `u.f` ?

4.2 - Analyse de dump

```
0x7FFE4000: 01 00 00 00 00 00 F0 3F
```

```c
typedef struct {
    int type;
    union {
        int i;
        double d;
    } value;
} Variant;
```

Questions:

1. Quel est `sizeof(Variant)` ?
2. Quelle est la valeur de `type` ?
3. Si `type == 1` signifie `double`, quelle est la valeur stockée ?
   Note: `0x3FF0000000000000` est la représentation IEEE 754 de `1.0`.

4.3 - Détection d'endianness

```c
union EndianCheck {
    int i;
    char c[4];
};

int is_little_endian(void) {
    union EndianCheck e;
    e.i = 1;
    return e.c[0] == 1;
}
```

Questions:

1. Expliquer pourquoi ce code fonctionne
2. Dessiner le layout mémoire de `e` pour little-endian
3. Dessiner le layout mémoire de `e` pour big-endian

5 Exercice 5 : Bit Fields

5.1 - Calcul de taille

```c
struct Flags1 {
    unsigned int a : 1;
    unsigned int b : 3;
    unsigned int c : 4;
};

struct Flags2 {
    unsigned int a : 1;
    unsigned int b : 3;
    unsigned int : 0;  // force alignment
    unsigned int c : 4;
};
```

Questions:

1. Quel est `sizeof(struct Flags1)` ?
2. Quel est `sizeof(struct Flags2)` ?
3. Expliquer l'effet de `: 0`

5.2 - Représentation binaire

```c
struct Status {
    unsigned int active : 1;
    unsigned int mode : 2;
    unsigned int priority : 3;
    unsigned int reserved : 2;
};

struct Status s = {1, 2, 5, 0};
```

Questions:

1. Représenter `s` en binaire (8 bits)
2. Quelle est la valeur hexadécimale du premier octet ?
3. Si on lit `s` comme un `unsigned char`, quelle valeur obtient-on ?

6 Exercice 6 : Structures et Allocation

6.1 - Analyse de code

```c
struct Person {
    char *name;
    int age;
};

struct Person *p = malloc(sizeof(struct Person));
p->name = "Alice";
p->age = 25;
// ...
free(p);
```

Questions:

1. Ce code est-il correct ? Justifier.
2. Proposer une version avec allocation de `name`
3. Écrire la fonction de libération correspondante

6.2 - Flexible Array Member

```c
typedef struct {
    size_t len;
    char data[];
} String;

String *s = malloc(sizeof(String) + 6);
s->len = 5;
strcpy(s->data, "Hello");
```

Questions:

1. Quel est `sizeof(String)` ?
2. Pourquoi alloue-t-on `sizeof(String) + 6` ?
3. Dessiner le layout mémoire de `s`

6.3 - Structure opaque

Soit le header:

```c
// stack.h
typedef struct Stack Stack;
Stack *stack_create(size_t capacity);
void stack_push(Stack *s, int value);
int stack_pop(Stack *s);
void stack_destroy(Stack *s);
```

Questions:

1. Pourquoi ne peut-on pas écrire `Stack s;` dans le code client ?
2. Pourquoi peut-on écrire `Stack *s;` ?
3. Proposer une implémentation de `struct Stack`

7 Exercice 7 : Forensics Avancé

7.1 - Structure complexe

```
0x7FFE5000: 01 00 00 00 00 00 00 00 10 FE 7F 00 00 00 00
            05 00 00 00 00 00 00 00
```

```c
typedef struct {
    int type;
    void *data;
    size_t size;
} Buffer;
```

Questions:

1. Calculer `sizeof(Buffer)` et identifier le padding
2. Extraire les valeurs de `type`, `data`, `size`
3. À quelle adresse pointe `data` ?

7.2 - Tableau dynamique de structures

```
0x00602000: 00 00 00 00 0A 00 00 00 01 00 00 00 14 00 00 00
            02 00 00 00 1E 00 00 00
```

```c
struct Item {
    int id;
    int value;
};

struct Item *items = malloc(3 * sizeof(struct Item));
```

Questions:

1. Reconstruire le contenu du tableau
2. Quel est l'offset de `items[1].value` par rapport à `items` ?
3. Écrire l'expression pour accéder à `items[2].id` via arithmétique de pointeurs